School of Future Tech

Case Study Report

on

PetMatch – Pet Adoption Portal Backend System

by

Jui Tawde
Roll Number: 150096725149
Cohort: Sam Altman
Academic Year: 2026–2027

Index

1. Introduction to the Case Study 3

2. Problem Statement / Case Background (Abstract) 3

3. Case Study Design 4

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study . . 6

5. Problem Statement / Case Study Implementation Details and Snapshots 7

6. Problem Statement / Case Study Results and Conclusion 16

7. References 18

1. Introduction to the Case Study

Pet adoption is an important process that connects animals in need of a home with people who are willing to provide them with care and shelter. However, managing pet information, adopter details, adoption applications, and application status manually can become difficult as the number of pets and applicants increases.

This case study focuses on the development of the **PetMatch – Pet Adoption Portal Backend System**, a RESTful backend application designed to manage pet adoption activities through structured APIs. The backend provides services for user authentication, pet information management, adoption application submission, application status management, and administrative operations.

The system is developed using Node.js and Express.js for server-side development, with MongoDB and Mongoose for data storage and database interaction. Authentication mechanisms involving JWT and Firebase Authentication support user identity and access management. Socket.IO provides real-time communication capabilities, while Firebase Cloud Messaging supports push notifications for application updates.

The project also includes a React-based frontend that communicates with the backend through API requests. The frontend allows users to interact with the pet adoption system through a graphical interface instead of directly accessing the APIs.

This case study demonstrates how backend technologies can be integrated to develop a modular, maintainable, and functional web application for managing pet adoption activities.

2. Problem Statement / Case Background (Abstract)

Background

Pet adoption platforms need to manage different types of information, including pet profiles, adopter accounts, adoption applications, and administrative decisions. Without a centralized system, it can become difficult to maintain consistent records, control access to sensitive operations, and communicate application updates to users.

A web-based backend system can address these challenges by providing standardized APIs for data management, authentication, and communication between the frontend and database. Role-based access control can also help restrict administrative operations to authorized users.

Problem Statement

To design and implement a modular backend system for a pet adoption portal that supports user registration and login, pet management, adoption application processing, administrative operations, and application status notifications.

The system should provide RESTful API endpoints, store information in a database, protect authenticated routes, and support communication between the frontend and backend.

Abstract

This case study presents the design and implementation of the PetMatch Pet Adoption Portal Backend System using Node.js, Express.js, MongoDB, and Mongoose.

The backend is organized into separate route modules responsible for authentication, pets, adoption applications, and administrative functionality. MongoDB stores application data, while Mongoose provides schemas and database interaction. Authentication and authorization mechanisms help protect restricted operations and manage access to resources.

The application supports pet listing and management, adoption application submission, application retrieval and status updates, and administrative actions. Socket.IO is integrated to support real-time communication, and Firebase services are used for authentication-related functionality and push notifications.

A React frontend provides the user interface through which users and administrators interact with the backend. The project is deployed online, allowing the application to be accessed through a web browser.

The implementation demonstrates the integration of REST APIs, database operations, authentication, authorization, and notification services in a full-stack web application.

3. Case Study Design

The PetMatch backend is designed using a modular architecture in which each major responsibility is handled by a separate component.

3.1 System Architecture

The system consists of the following components:

1. Frontend Layer

The React frontend provides the user interface for registration, login, browsing available pets, submitting adoption applications, and viewing relevant application information. It communicates with the backend using HTTP requests through Axios.

2. Backend Server Layer

Node.js provides the JavaScript runtime, while Express.js manages HTTP requests, middleware, and API routes. The backend server acts as the central layer connecting the frontend, database, authentication services, and notification functionality.

3. Authentication and Authorization Layer

The authentication system supports user identity verification and JWT-based authenticated requests. Authorization checks help restrict protected operations to users with the required permissions, including administrative operations.

4. Database Layer

MongoDB stores application data, while Mongoose defines schemas and supports database queries and document operations. The database manages information associated with pets, users or adopters, and adoption applications.

5. Real-Time Communication Layer

Socket.IO enables real-time communication between connected clients and the server for supported application events.

6. Notification Layer

Firebase Cloud Messaging supports push notifications, including adoption application status updates when the relevant notification workflow is triggered.

3.2 System Workflow

The application follows the workflow below:

1. A user opens the PetMatch frontend.
2. The user registers or logs in through the authentication system.
3. The frontend sends API requests to the Express backend.
4. The backend validates incoming requests and checks authentication or authorization when required.
5. Relevant routes and controllers process the requested operation.
6. The backend interacts with MongoDB through Mongoose when data needs to be created, retrieved, updated, or deleted.
7. Adoption applications are submitted and processed through the application-management functionality.
8. Authorized administrative operations can be used to manage pets and update application statuses.
9. Supported real-time and push-notification services communicate relevant updates to connected clients or registered devices.
10. The backend returns an HTTP response to the frontend, which displays the result to the user.

3.3 Main Functional Modules

Authentication Module

Handles registration and login functionality and supports authenticated access to protected resources.

Pet Management Module

Manages pet information, including retrieval of available pet records and authorized creation, modification, or deletion operations.

Adoption Application Module

Supports submitting adoption applications, retrieving application information, and processing application status changes.

Administration Module

Provides administrative API functionality for operations that require elevated permissions.

Database Module

Establishes the MongoDB connection and supports persistent storage through Mongoose models.

Real-Time Communication and Notifications Module
Uses Socket.IO for supported real-time events and Firebase Cloud Messaging for push-notification functionality.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

4.1 Methods and Techniques Applied

1. RESTful API Design
The backend uses HTTP methods to represent operations on application resources.

- GET is used to retrieve information.
- POST is used to submit data or create resources.
- PUT is used to update existing resources where supported.
- DELETE is used to remove resources where supported.

The API routes are organized by functionality to improve code readability and maintainability.

2. Modular Backend Architecture
The backend separates API route definitions and application responsibilities into modules. This makes it easier to understand, test, maintain, and extend the application.

3. Database Management
MongoDB is used as the database, and Mongoose supports schema-based data modelling and database operations. This allows the application to store and retrieve pet records and adoption-related information.

4. Authentication Using JWT
JSON Web Tokens (JWT) support authenticated requests. After authentication, a client can include a token in the Authorization header using the Bearer token format. The backend can verify the token before allowing access to protected endpoints.

5. Authorization and Role-Based Access
Authorization checks help restrict operations according to user permissions. Administrative operations, such as managing pet records or processing applications, should only be accessible to appropriately authorized users.

6. Real-Time Communication
Socket.IO supports communication between the backend and connected clients for implemented real-time events. This can be used to communicate changes without requiring users to manually refresh the application.

7. Push Notifications
Firebase Cloud Messaging supports push notifications for events such as adoption application status updates, provided that the device or browser is registered and notification permissions and delivery configuration are correctly set up.

8. Error Handling and Validation

The backend processes incoming requests and returns HTTP responses. Input validation, authentication checks, and appropriate error responses help improve reliability and prevent invalid operations.

4.2 Technology Stack Used for the Case Study

Programming Language: JavaScript

Runtime Environment: Node.js

Backend Framework: Express.js

Database: MongoDB

Database Object Modelling: Mongoose

Authentication Technologies: JWT and Firebase Authentication

Real-Time Communication: Socket.IO

Push Notifications: Firebase Cloud Messaging

API Documentation: Swagger UI, where configured in the backend

Frontend Integration: React.js and Axios

Middleware and Supporting Packages: CORS, Express JSON middleware, and Morgan for HTTP request logging.

Development Environment: Visual Studio Code and npm.

Version Control: Git and GitHub.

Deployment Platforms: Render for the backend and the configured frontend hosting platform.

5. Problem Statement / Case Study Implementation Details and Snapshots

The PetMatch project is implemented as a full-stack application, with a React frontend communicating with a Node.js and Express backend.

5.1 Backend Server Configuration

The backend server is initialized using Node.js and Express.js. An HTTP server is created to support Express request handling and Socket.IO communication.

Middleware is configured to process JSON request bodies, handle cross-origin requests, and log incoming HTTP requests. The backend also defines a health-check endpoint that can be used to verify server availability.

The server uses an environment-provided port when deployed, allowing it to run on the hosting platform's assigned port.

Main implementation file: `backend/server.js`

5.2 Database Connection

MongoDB is used for persistent storage. Mongoose is responsible for connecting the backend to the database and supporting database operations through defined models.

Database configuration is handled separately from the main server logic. Connection settings are provided through environment variables so that deployment credentials do not need to be hardcoded into the application.

The database connection must be successful before the backend starts accepting requests that depend on the database.

Main implementation area: Backend database connection module and Mongoose models.

5.3 API Route Organization

The backend organizes functionality into separate API route groups.

- `/api/health` — checks whether the backend is responding.
- `/api/auth` — handles authentication-related requests.
- `/api/pets` — handles pet-related operations.
- `/api/applications` — handles adoption application operations.
- `/api/admin` — provides administrative functionality.
- `/api/docs` — exposes API documentation through Swagger UI, where configured.

The exact supported operations depend on the route and controller implementations.

5.4 Authentication and Authorization

The authentication module provides registration and login functionality. JWT-based authentication supports protected API requests by allowing the frontend to send a token in the Authorization header.

Firebase Authentication is also integrated into the project's authentication workflow. Authentication and authorization checks help control access to protected resources and administrative operations.

Authentication is important because operations such as changing pet information or processing adoption applications should not be available to every unauthenticated visitor.

5.5 Pet Management

The pet-management module provides API functionality for working with pet information.

Users can retrieve pet information through the frontend, while authorized administrative operations support managing pet records. The database stores the relevant information so that records can be retrieved across requests rather than existing only in the browser.

5.6 Adoption Application Management

The adoption application module supports the process through which a user submits an application for a pet.

The backend handles application-related requests, stores application information, and provides functionality for retrieving applications and updating their statuses through authorized operations.

This module connects the adopter-facing functionality with the administrative workflow used to process applications.

5.7 Real-Time Communication and Notifications

Socket.IO is integrated into the backend to support real-time communication for implemented events.

Firebase Cloud Messaging supports push notifications. For example, an application status update can trigger a notification workflow that informs the user of the change, provided the recipient has a valid registration token and the notification delivery configuration is working.

The visibility of browser notifications also depends on browser permissions, service-worker configuration, and device settings.

5.8 Frontend and Backend Integration

The frontend communicates with the backend through Axios. The API configuration defines the backend base URL and supports environment variables for deployment-specific configuration.

The frontend sends requests to the backend for authentication, pet information, and adoption application operations. The backend processes the requests and returns responses that the frontend uses to update the interface.

Cross-origin resource sharing (CORS) is configured on the backend to allow requests from the intended frontend origin. Deployment environment variables must contain the correct frontend URL so that browser requests are not blocked by an origin mismatch.

5.9 Deployment Details

The project source code is maintained in GitHub. The frontend and backend are deployed as separate services.

The backend is hosted on Render and exposes API endpoints over HTTPS. The frontend is hosted separately and communicates with the backend using its deployed URL.

The deployed backend's health-check endpoint can be used to verify that the server responds. Successful access to the health-check endpoint confirms server availability, although individual API routes and database-dependent operations must be tested separately.

Live project: https://pet-adoption-portal-1.onrender.com/

GitHub repository: https://github.com/juitawde/pet-adoption-portal

Backend health check: https://petmatch-backend-qi7c.onrender.com/api/health

5.10 Snapshots

The snapshots below were captured from the running application, the project codebase, the database and the hosting dashboard. Sensitive details — phone numbers stored in the database, the demo login credentials, the Render service ID and the local computer username — are hidden with red pen marks.

Snapshot 1: PetMatch Homepage

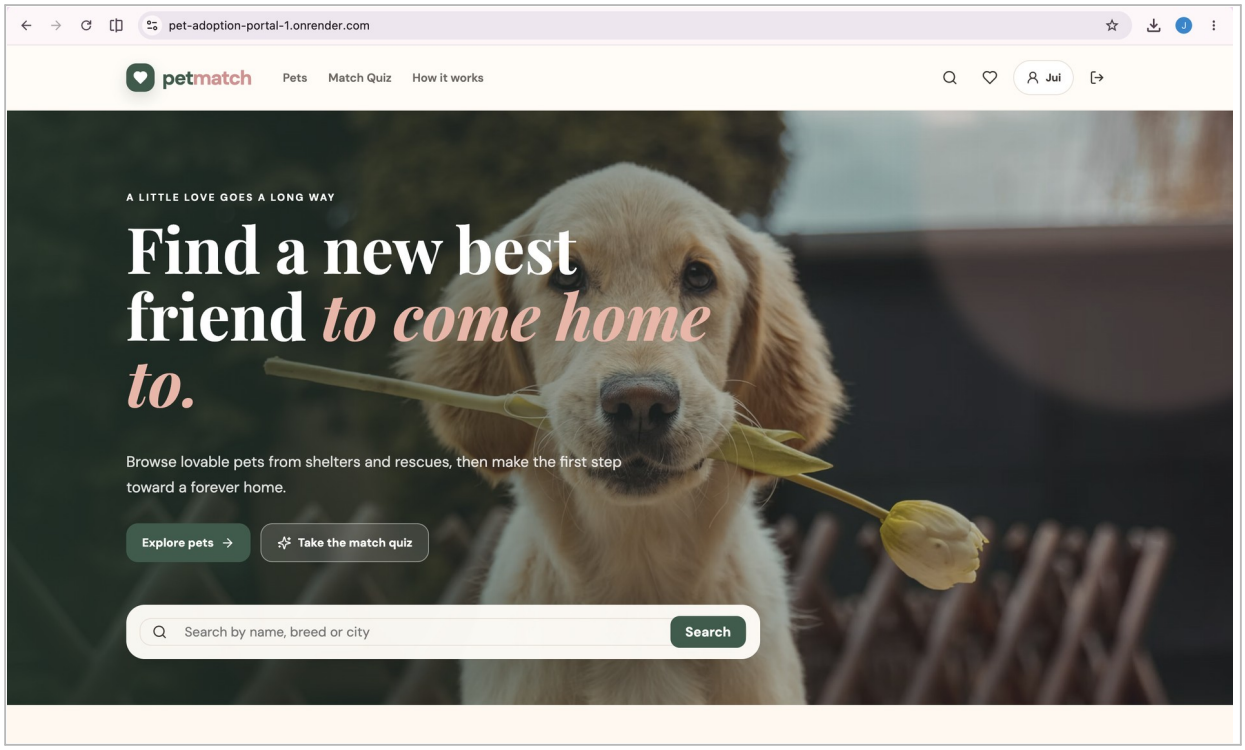


Figure 1: Hero section of the deployed PetMatch homepage.

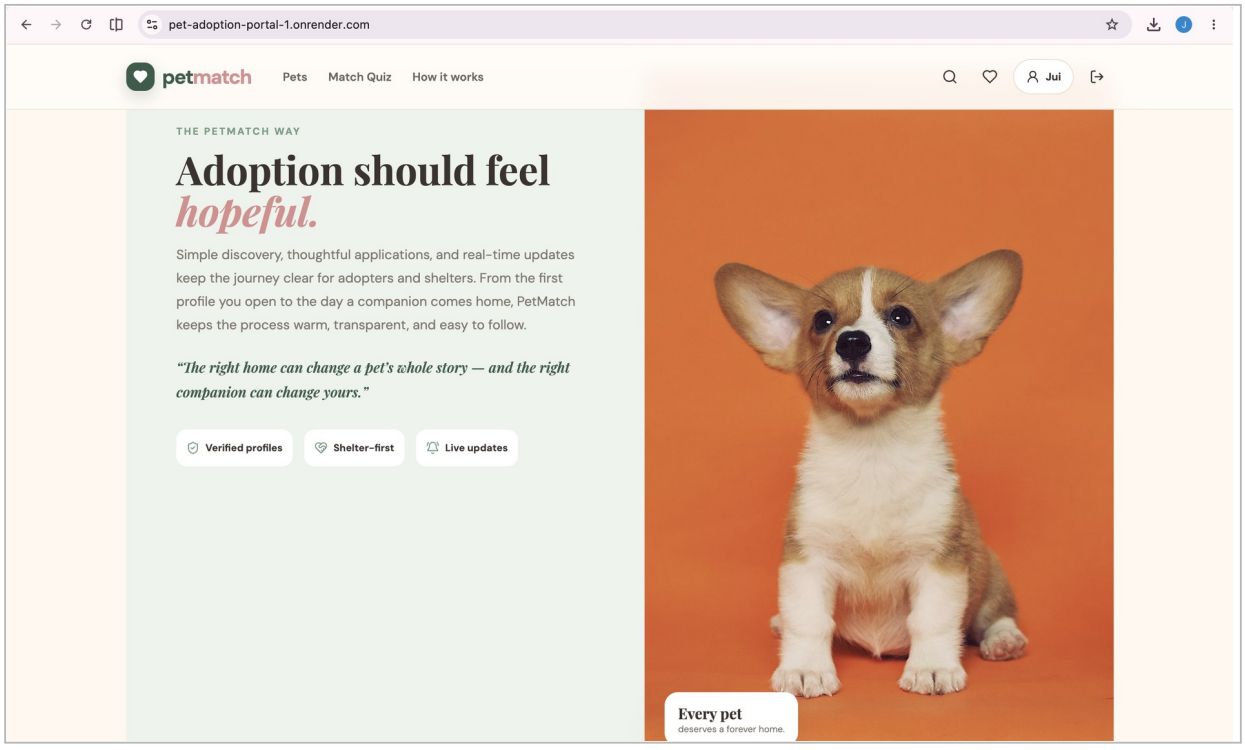


Figure 2: "The PetMatch Way" section of the homepage.

The first figure shows the deployed frontend at `pet-adoption-portal-1.onrender.com`. The top navigation bar links to the **Pets** listing, the **Match Quiz** and the **How it works** page, and provides search, favourites, the signed-in user's profile button and a logout button. The hero section presents the headline "Find a new best friend to come home to", with an **Explore pets** button, a

Take the match quiz button and a search bar that lets visitors look for pets by name, breed or city.

The second figure shows the section that explains the approach of the portal, with the highlights **Verified profiles**, **Shelter-first** and **Live updates**. Together these screens form the user-facing side of the REST API described in this report.

Snapshot 2: Backend Project Structure

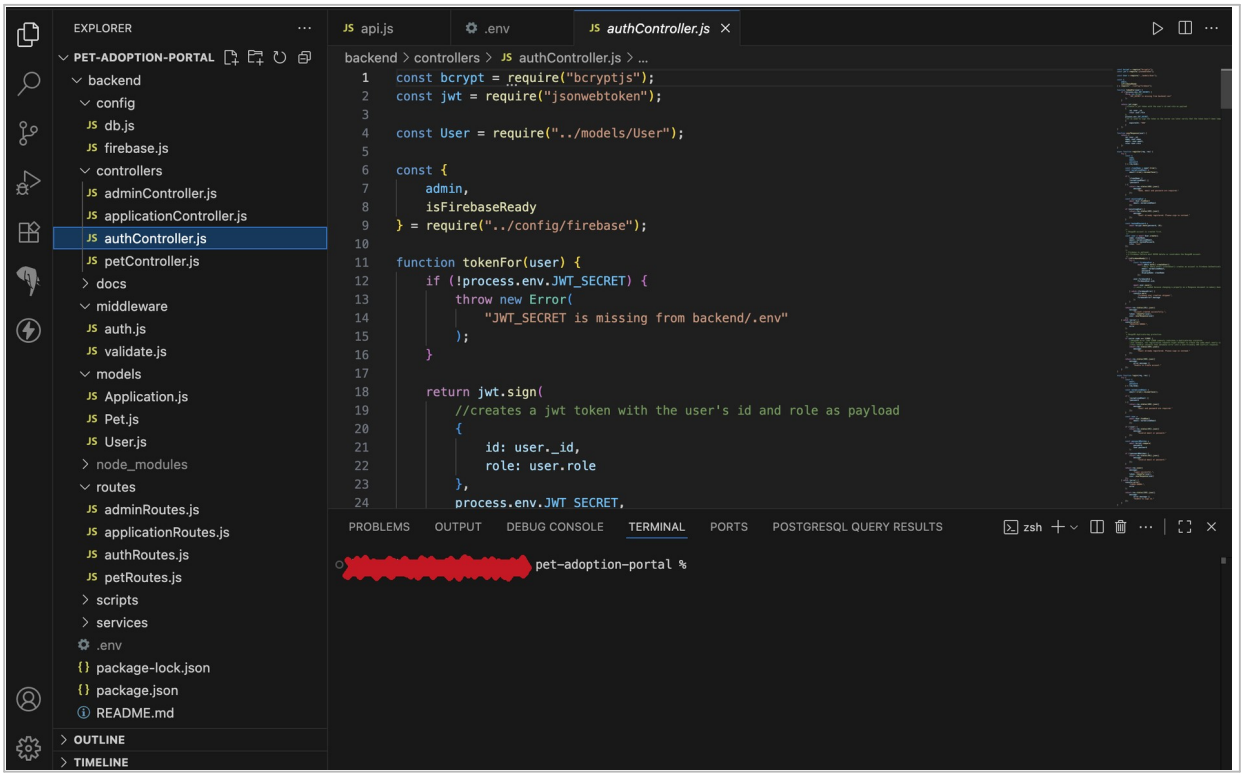


Figure 3: Backend folder structure in Visual Studio Code, with `authController.js` open.

The Explorer panel shows how the `backend` folder is organised into separate folders by responsibility:

- `config` — `db.js` (database connection) and `firebase.js` (Firebase setup).
- `controllers` — `adminController.js`, `applicationController.js`, `authController.js` and `petController.js`.
- `middleware` — `auth.js` (authentication and authorization checks) and `validate.js` (request validation).
- `models` — Mongoose models `Application.js`, `Pet.js` and `User.js`.
- `routes` — `adminRoutes.js`, `applicationRoutes.js`, `authRoutes.js` and `petRoutes.js`.
- Other items — `docs`, `scripts`, `services`, the `.env` environment file (its contents are not shown), `package.json`, `package-lock.json` and `README.md`.

The editor shows `authController.js`. It imports `bcryptjs` for password hashing, `jsonwebtoken` for JWT handling, the `User` model and the Firebase configuration. The `tokenFor` function stops with an error if `JWT_SECRET` is missing from the environment and otherwise signs a JWT whose payload contains the user's id and role. The local computer username that appeared in the terminal is hidden.

Snapshot 3: Backend Server Configuration

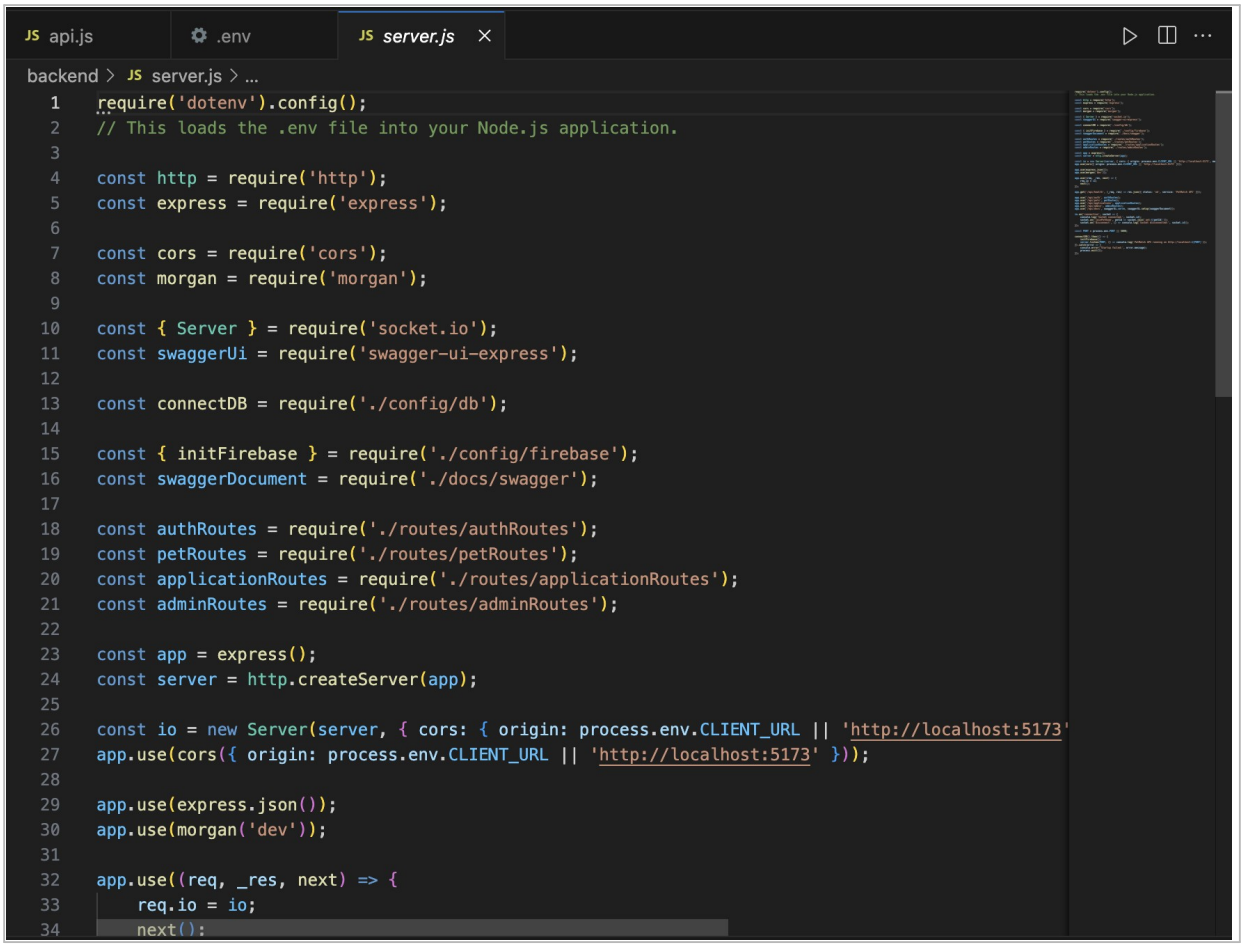


Figure 4: Beginning of `backend/server.js`*: imports, Express and Socket.IO initialization, and middleware.*

`server.js` is the entry point of the backend. It loads environment variables with `dotenv` and imports `http`, `express`, `cors`, `morgan`, the Socket.IO `Server`, `swagger-ui-express`, the database connector (`./config/db`), the Firebase initializer (`./config/firebase`), the Swagger document (`./docs/swagger`) and the four route modules (`authRoutes`, `petRoutes`, `applicationRoutes` and `adminRoutes`).

An Express application is created and wrapped in an HTTP server so that Express and Socket.IO can share the same port. Socket.IO and CORS are both configured to accept requests from the origin given in the `CLIENT_URL` environment variable, falling back to `http://localhost:5173` during local development. The JSON body parser and the `morgan('dev')` request logger are registered next, followed by a small middleware that attaches the Socket.IO instance to each request as `req.io`, so route handlers can emit real-time events.

Snapshot 4: API Health Check



Figure 5: Response of the deployed backend's `/api/health` *endpoint.*

Opening `https://petmatch-backend-qi7c.onrender.com/api/health` in the browser returns the JSON response `{"status":"ok","service":"PetMatch API"}`. This confirms that the deployed backend server is running and reachable over HTTPS. As noted in Section 5.9, this check confirms server availability only; database-dependent routes are tested separately through the application.

Snapshot 5: Database Integration

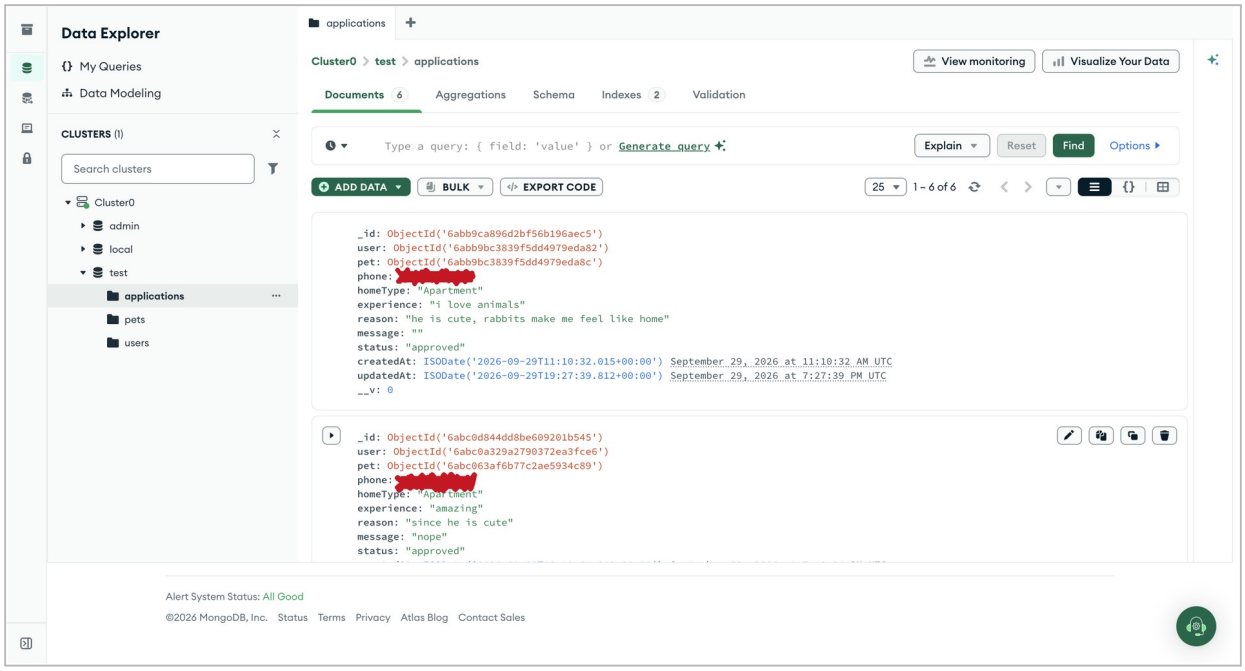


Figure 6: MongoDB Atlas Data Explorer showing the `applications` collection (phone numbers hidden).

MongoDB Atlas Data Explorer shows `Cluster0` with a `test` database that contains three collections: `applications`, `pets` and `users`. The `applications` collection is open and currently holds six documents.

Each application document stores references to the applicant (`user`) and the pet (`pet`) as ObjectIds, together with the form details `phone`, `homeType`, `experience`, `reason` and `message`, the processing `status` (for example `approved`), the timestamps `createdAt` and `updatedAt`, and the Mongoose version key `__v`. This demonstrates that adoption applications are stored persistently and linked to the related user and pet records. The applicants' phone numbers are hidden with red pen marks.

Snapshot 6: Authentication and Adoption Application Workflow

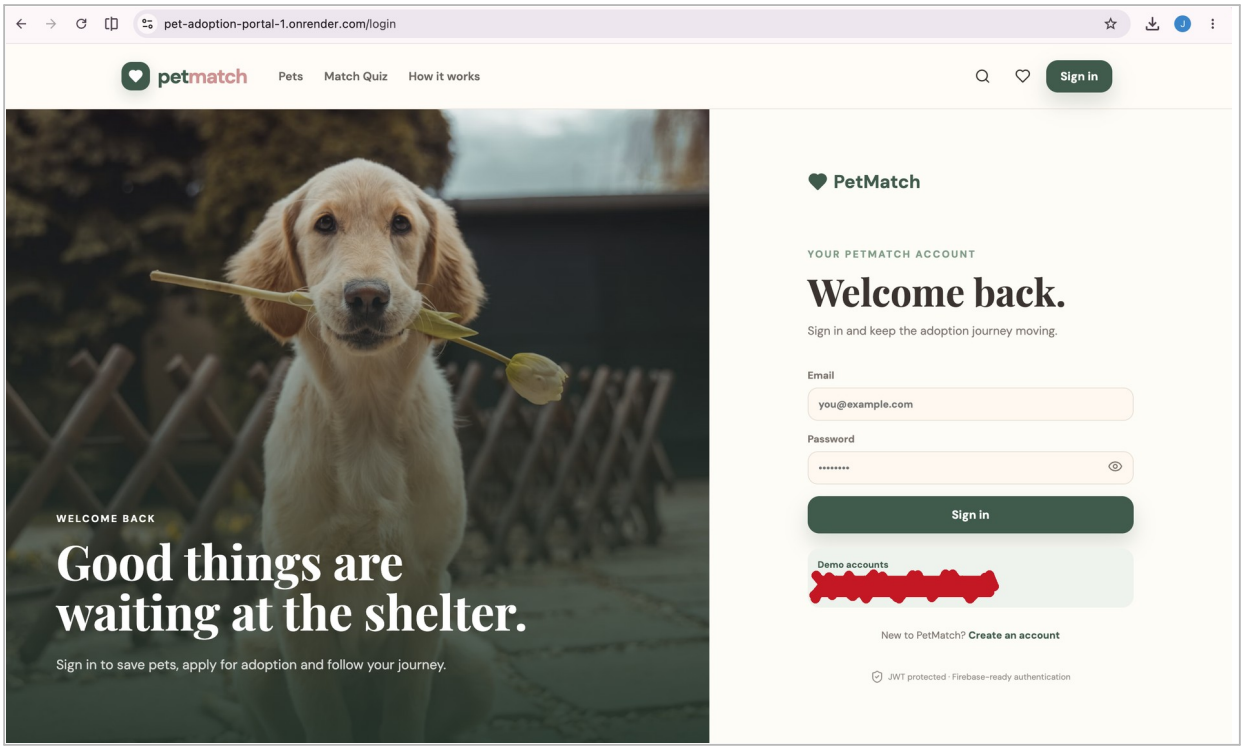


Figure 7: Login page of PetMatch (demo account credentials hidden).

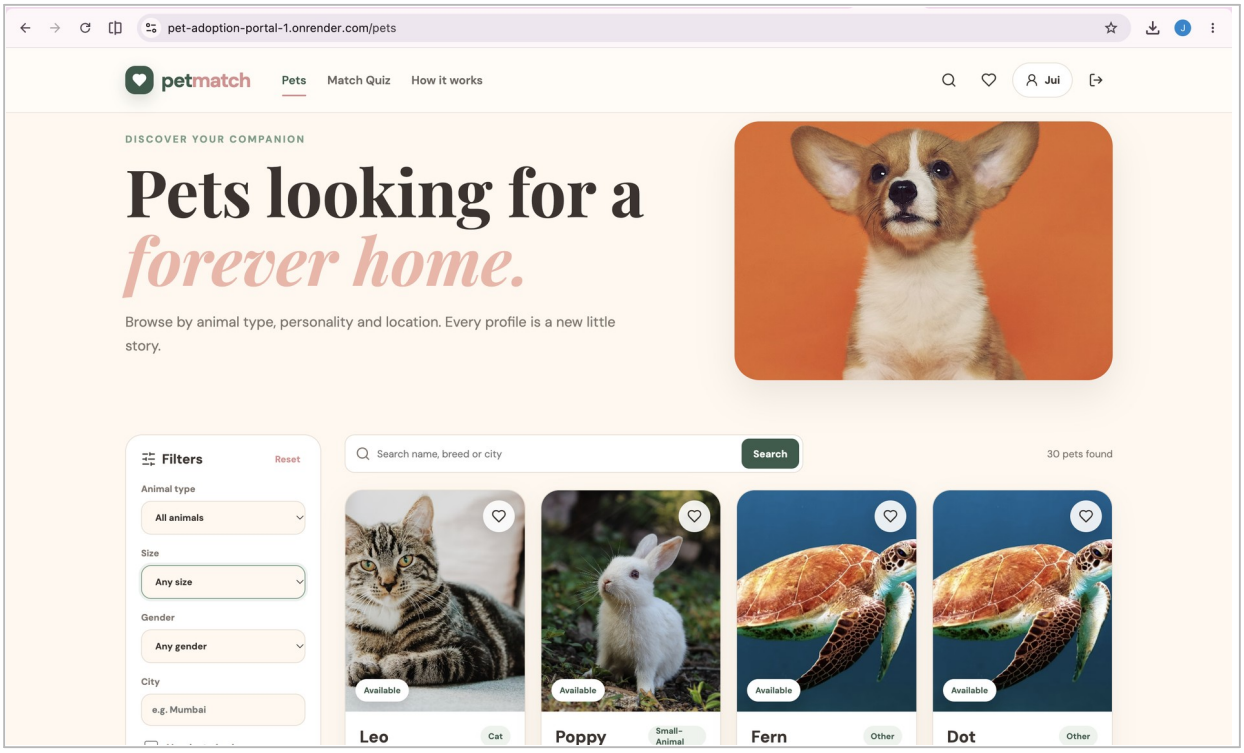


Figure 8: Pet listing page with filters, search and availability status.

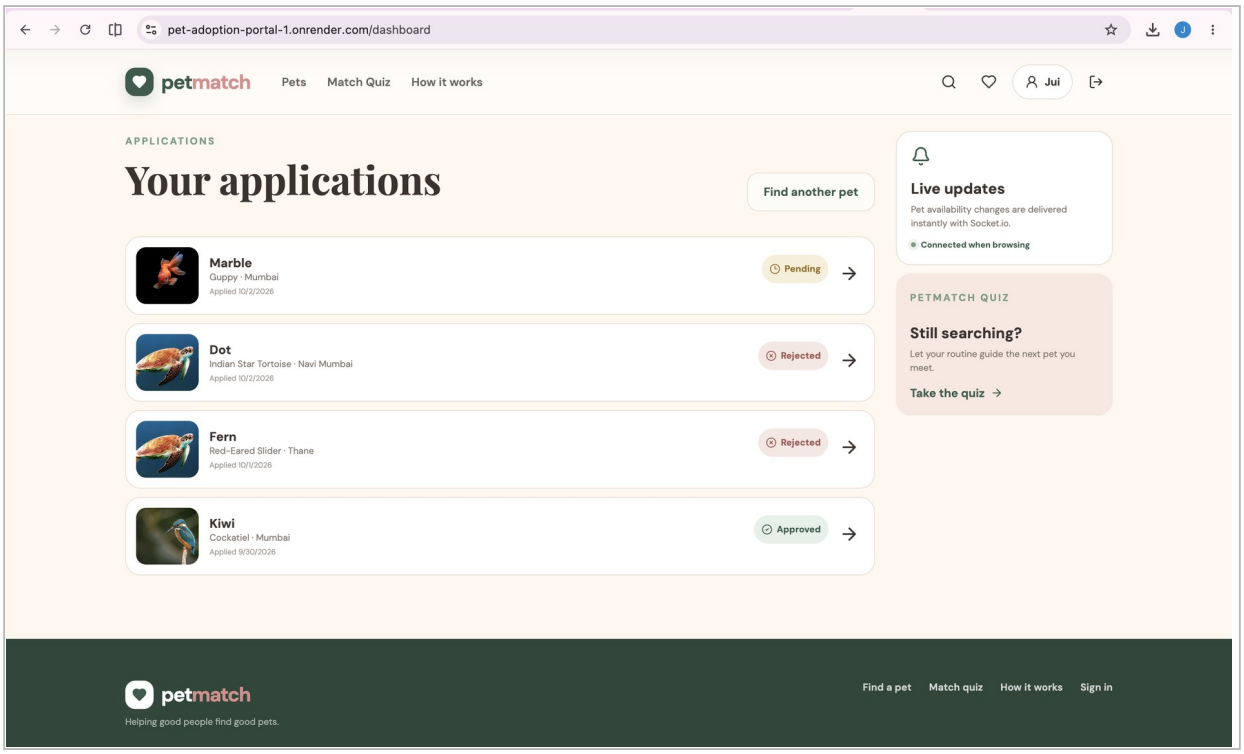


Figure 9: "Your applications" dashboard showing application statuses.

Login. The sign-in page asks for an email address and a password and offers a link to create a new account. A note at the bottom of the form states that the page uses JWT-protected, Firebase-ready authentication. The demo account credentials shown on this page are hidden.

Pet listing. After signing in, the **Pets** page lists the available animals (30 pets found at the time of capture). Visitors can filter by animal type, size, gender and city, or search by name, breed or city. Each pet card shows its name, animal type, an **Available** badge and a heart button for saving it to favourites; the first row shows Leo (Cat), Poppy (Small-Animal), and Fern and Dot (Other).

Application tracking. The dashboard lists the signed-in user's adoption applications with the pet's name, breed, city, application date and current status: Marble (Guppy, Mumbai) is **Pending**, Dot (Indian Star Tortoise, Navi Mumbai) and Fern (Red-Eared Slider, Thane) are **Rejected**, and Kiwi (Cockatiel, Mumbai) is **Approved**. A **Live updates** panel notes that pet availability changes are delivered instantly with Socket.IO, and a **Find another pet** button and a Match Quiz prompt help the user continue searching.

Snapshot 7: Deployment on Render

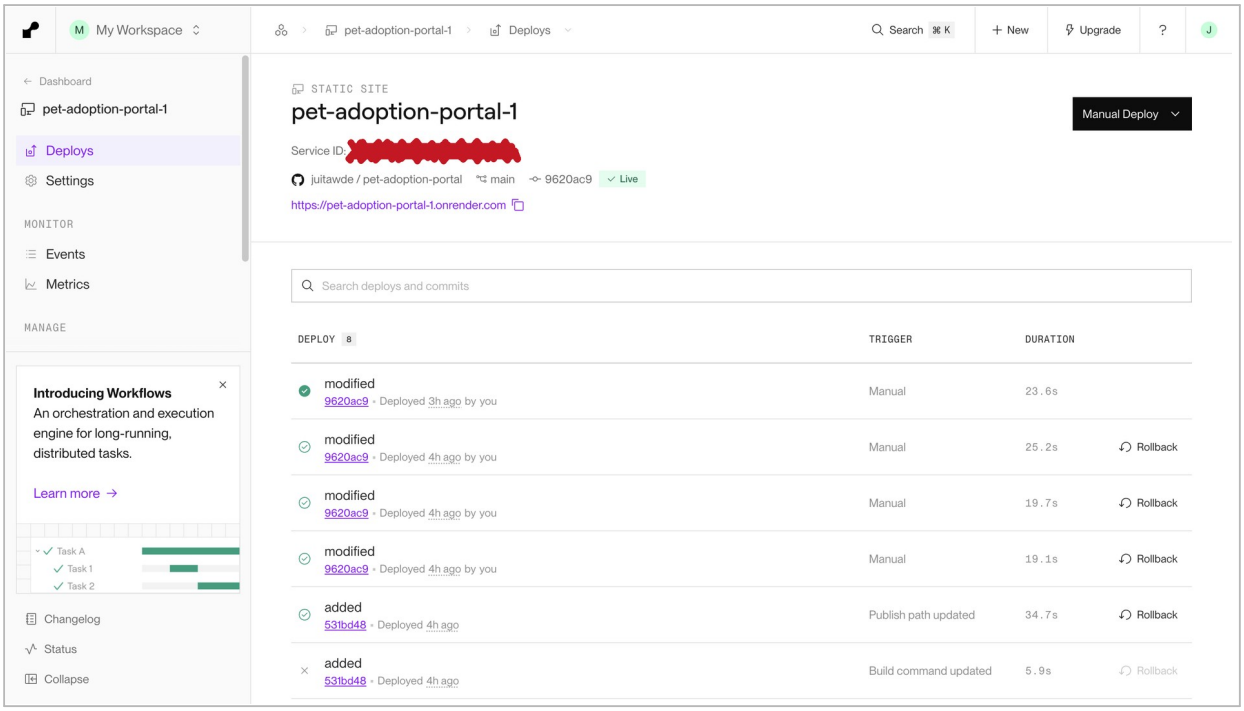


Figure 10: Render dashboard for the deployed `pet-adoption-portal-1` *static site (service ID hidden).*

The Render dashboard shows the `pet-adoption-portal-1` static site with a green **Live** status. It is connected to the GitHub repository `juitawde/pet-adoption-portal`, deploys from the `main` branch (commit `9620ac9`) and is served at `https://pet-adoption-portal-1.onrender.com`. The **Deploys** list shows eight deploys; those visible were triggered manually or by a settings change (such as an updated build command or publish path), with durations of roughly 6 to 35 seconds. The service ID is hidden with a red pen mark.

6. Problem Statement / Case Study Results and Conclusion

Findings

The PetMatch backend demonstrates how multiple server-side technologies can be integrated into a single application to support pet adoption activities.

- The Express.js server provides an organized structure for handling HTTP requests and API routes.
- RESTful endpoints allow the frontend to communicate with backend services.
- MongoDB and Mongoose support persistent storage and structured database interactions.
- JWT-based authentication and authorization checks support protected operations.
- Separate route modules help organize authentication, pet management, adoption applications, and administrative functionality.
- Socket.IO provides the infrastructure for supported real-time communication.
- Firebase Cloud Messaging supports push-notification workflows when the required device registration and delivery configuration are available.

- Environment variables allow deployment-specific settings, such as database credentials and allowed frontend origins, to be configured outside the application source code.
- The deployed backend health-check endpoint provides a way to verify server availability.

The overall functionality of the application depends on correct database configuration, valid authentication credentials, appropriate user permissions, and successful integration of external services.

Conclusion

This case study presents the design and implementation of the PetMatch Pet Adoption Portal Backend System using Node.js, Express.js, MongoDB, Mongoose, JWT, Firebase, and Socket.IO.

The project demonstrates the role of backend development in managing application data, providing RESTful APIs, protecting restricted operations, and supporting communication between the frontend and server. Its modular organization separates major responsibilities and provides a foundation for future improvements.

The deployment process also demonstrates the importance of configuring environment variables, database connectivity, CORS, and frontend API URLs correctly when moving an application from local development to production.

Overall, PetMatch provides a practical example of integrating backend services into a web-based pet adoption platform. Further development could include more extensive automated testing, stronger input validation, improved monitoring, and additional adoption-management features.

7. References

1. Node.js Official Documentation — https://nodejs.org/docs/latest/api/
2. Express.js Official Documentation — https://expressjs.com/
3. MongoDB Official Documentation — https://www.mongodb.com/docs/
4. Mongoose Official Documentation — https://mongoosejs.com/docs/
5. JSON Web Tokens — https://jwt.io/introduction
6. Firebase Authentication Documentation — https://firebase.google.com/docs/auth
7. Firebase Cloud Messaging Documentation — https://firebase.google.com/docs/cloud-messaging
8. Socket.IO Official Documentation — https://socket.io/docs/v4/
9. Axios Documentation — https://axios-http.com/docs/intro
10. Swagger UI Documentation — https://swagger.io/tools/swagger-ui/
11. Render Documentation — https://render.com/docs
12. Vercel Documentation — https://vercel.com/docs
13. PetMatch GitHub Repository — https://github.com/juitawde/pet-adoption-portal
14. PetMatch Live Application — https://pet-adoption-portal-1.onrender.com/